Author: Uzair Khan

Jump task: Energy Efficiency of AI Benchmarks and Large Language Model Pipelines for Wattlytics Integration

1.Experimental Setup

The hints and guide helped me immensely in setting up the environment. First of all, I chose **cip7c2** amongst cip7 which have the required **NVIDIA GeForce RTX 3070** GPU (8GB VRAM) for this task. The hardware imposes a strict Total Board Power (TDP) limit of 220W.

1.1. Software Stack

- **CUDA & Compiler:** CUDA Driver 13.3, GCC 14.2.
- **Python Environment:** Python 3.11, PyTorch 2.12.1+cu130.
- **Custom Kernels:** Mamba-SSM and causal-conv1d (v1.6.2) were built from source locally in I figured out that the models don't take much space so instead of ciptmp directory I used my local directory. This made my setup persistent and I didn't face any storage issue thankfully. Rest of the setup I built as per the hint.

1.2. Evaluated Models

To isolate the impact of model architecture, models with comparable parameter counts were selected:

- **Baseline Transformers :** EleutherAI/pythia-70m, EleutherAI/pythia-160m.
- **State-Space Model:** state-spaces/mamba2-130m.

All models were loaded in bfloat16 precision to maximize the available 8GB VRAM capacity.

2. Measurement Methodology

Accurate hardware profiling in PyTorch requires mitigating the asynchronous nature of GPU execution and the coarse sampling rate of standard monitoring tools.

- **Time-Based Hardware Warm-up:** Before recording metrics, a sustained 1-second load was executed to force the RTX 3070 from its idle state (P8) into its maximum performance state (P0). This prevents skewed efficiency readings caused by GPU wake-up latency.

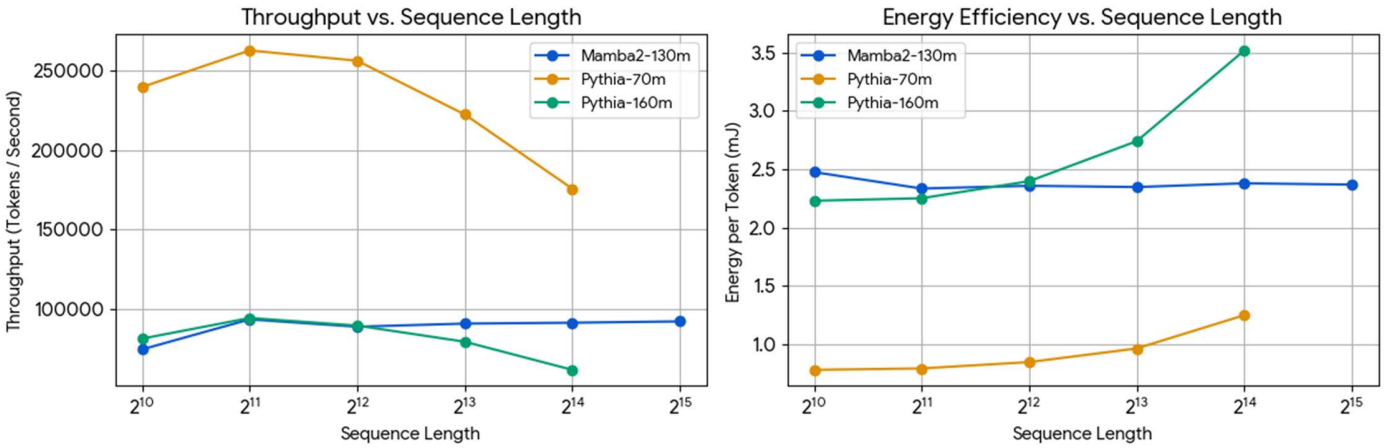
- **Adaptive Iteration Calibration:** Single forward passes for short sequences (e.g., 512–1024 tokens) complete in less than 100 milliseconds, which is faster than NVML's hardware polling frequency (50-60Hz). The benchmark dynamically calculated the required number of queued forward passes to ensure the workload ran for at least 2.0 seconds.

- **Explicit Asynchronous Polling:** Using torch.cuda. Event markers, the CPU was trapped in an explicit while loop during kernel execution, polling NVML for power consumption every 5ms. This ensured that only active execution power was averaged, filtering out CPU overhead.

3. Key Findings and Insights

The parameter sensitivity scaled sequence lengths from 1K to 32K and batch sizes from 1 to 32. The results expose a profound architectural divergence between standard attention mechanisms and State-Space Models.

3.1. Identifying the Crossover Point



By comparing Mamba2-130m with Pythia-160m (similar parameter counts) at a Batch Size of 1, the theoretical crossover point is clearly visible at **4096** tokens:

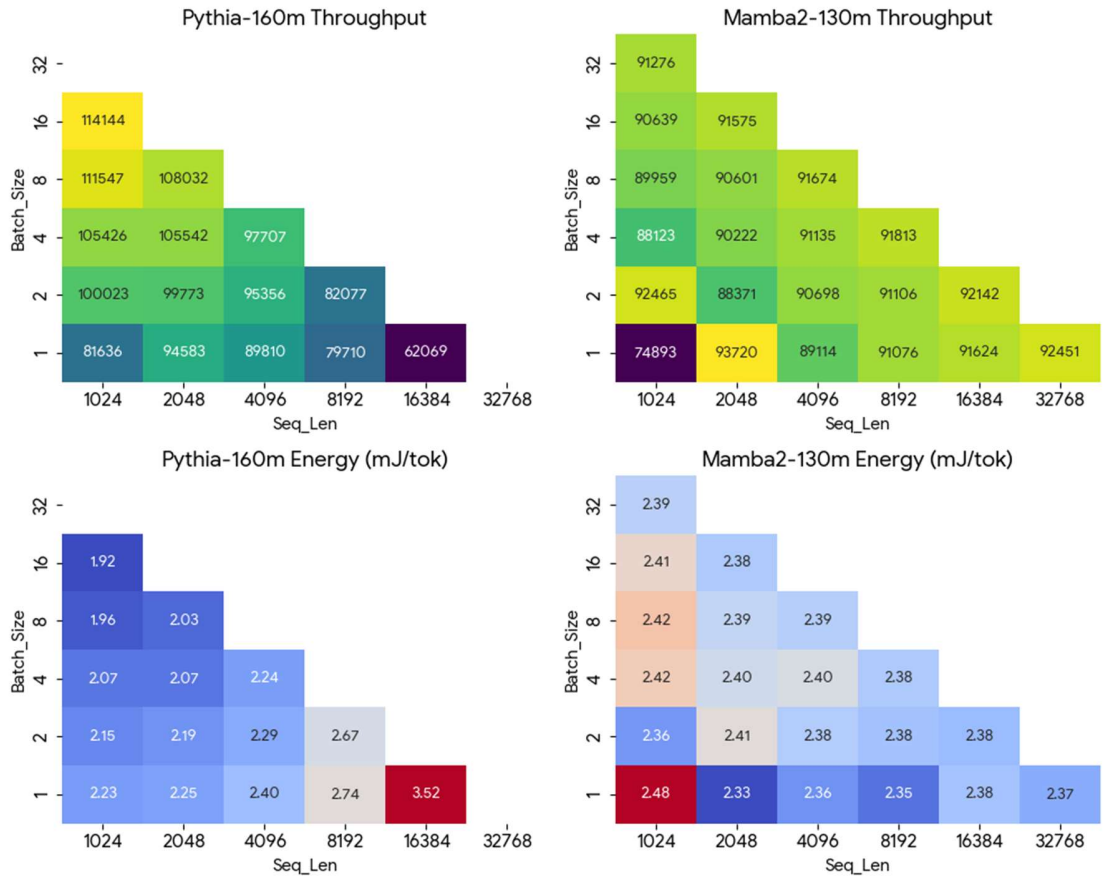
Author: Uzair Khan

- **Short Context (< 4K):** Pythia-160m holds a slight throughput advantage (94k vs. 93k tok/s at 2K tokens) and better energy efficiency (2.25 vs 2.33 mJ/tok). This indicates that the quadratic penalty is negligible for short sequences, and standard Transformers benefit from mature, highly optimized hardware attention kernels.

- **The Inflection (4K):** Performance intersects. Pythia drops to 89k tok/s, matching Mamba2.

- **Long Context (> 4K):** Mamba2 asserts dominance. At 16K tokens, Pythia's throughput collapses to 62k tok/s with a massive energy spike to 3.52 mJ/tok. Conversely, Mamba2 maintains a perfectly flat profile, sustaining 91k tok/s at 2.37 mJ/tok regardless of length.

3.2. The Memory Boundary (OOM Survival)



The heatmaps map the strict 8GB VRAM boundaries of the RTX 3070:

- **Quadratic:** Pythia models exhibit a strict diagonal failure line. Notably, Pythia-70m and Pythia-160m fail at the exact same coordinates (e.g., Batch 1 at 32K, Batch 2 at 16K). This

proves that at long contexts, the $O(N^2)$ attention matrix completely dominates memory consumption, rendering model parameter size irrelevant.

- **Linear:** Mamba2 successfully executes workloads that crash the baseline models. By compressing context into a fixed-size vector, it avoids the $O(N^2)$ memory explosion, surviving Batch 1 at 32K and Batch 2 at 16K.

3.3. Linear vs. Quadratic Power Draw

Validation of the benchmark script is confirmed by the Avg_Power_W metric, which remained pinned between 216W and 219W across almost all runs, proving the GPU reached its P0 state limits. The core difference lies in how that power was utilized:

Mamba2 exhibits a completely constant energy profile (2.33 to 2.41 mJ/tok) across the entire heatmap. Because its state-space update is O(1) per step, the GPU performs the exact same amount of work per token. Pythia's energy consumption, however, balloons from 2.22 mJ/tok to 3.52 mJ/tok as sequences grow. The GPU is forced to pull increasingly massive attention matrices from VRAM, burning energy on memory bandwidth rather than useful compute.

4. Assumptions and Limitations

- **Single-Stage Inference:** As instructed, inference was treated as a single stage, measuring raw parallel forward passes corresponding to the "Prefill" phase of LLM execution. The evaluation does not separate the auto regressive Decode phase.

- **Decode Implications:** While Mamba2 already demonstrates a clear crossover point in prefill, its true hardware advantage manifests in the Decode phase. Transformers suffer a massive memory-bandwidth bottleneck during generation because they must iteratively fetch an O(N) Key-Value (KV) cache for every token. Mamba's fixed O(1) state entirely bypasses this KV cache requirement, meaning the observed energy efficiency gap would theoretically widen significantly during long-context token generation.